



Foundation University Islamabad

School of Science and Technology

Name: Hunaina Yasir

Subject: Oop Lab

Roll Number: 073

Section: B

LAB NO. 8

Task1. Student and Employee Record System

Create a C++ program using hybrid inheritance to manage a system where a person can be both a student and an employee (e.g., an intern).

- Base class: Person (attributes: name, age)
- Derived classes: Student (attribute: rollNo), Employee (attribute: employeeId)
- Final class: StudentIntern (inherits from both Student and Employee) Implement functions to input and display data from each class.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Person {
public:
    string name;
    int age;
    void getPerson() {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Age: ";
        cin >> age;
    }
};

class Student : virtual public Person {
public:
    int rollNo;
    void getStudent() {
```

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Person {
6  public:
7      string name;
8      int age;
9      void getPerson() {
10         cout << "Enter Name: ";
11         cin >> name;
12         cout << "Enter Age: ";
13         cin >> age;
14     }
15 };
16
17 class Student : virtual public Person {
18 public:
19     int rollNo;
20     void getStudent() {
21         cout << "Enter Roll No: ";
22         cin >> rollNo;
23     }
24 };
25
26 class Employee : virtual public Person {
27 public:
28     int employeeId;
29     void getEmployee() {
30         cout << "Enter Employee ID: ";
31         cin >> employeeId;
32     }
33 };
34
35 class StudentIntern : public Student, public Employee {
```

```

        cout << "Enter Roll No: ";
        cin >> rollNo;
    }
};

class Employee : virtual public Person {
public:
    int employeeId;
    void getEmployee() {
        cout << "Enter Employee ID: ";
        cin >> employeeId;
    }
};

class StudentIntern : public Student, public Employee {
public:
    void display() {
        cout << "\n--- Intern Details ---" << endl;
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "Employee ID: " << employeeId << endl;
    }
};

int main() {
    StudentIntern si;
    si.getPerson();
    si.getStudent();
    si.getEmployee();
    si.display();
    return 0;
}

```

OUTPUT:

```

Enter Name: lia
Enter Age: 45
Enter Roll No: 23
Enter Employee ID: 678

--- Intern Details ---
Name: lia
Age: 45
Roll No: 23
Employee ID: 678

```

```

34
35 class StudentIntern : public Student, public Employee {
36 public:
37     void display() {
38         cout << "\n--- Intern Details ---" << endl;
39         cout << "Name: " << name << endl;
40         cout << "Age: " << age << endl;
41         cout << "Roll No: " << rollNo << endl;
42         cout << "Employee ID: " << employeeId << endl;
43     }
44 };
45
46 int main() {
47     StudentIntern si;
48     si.getPerson();
49     si.getStudent();
50     si.getEmployee();
51     si.display();
52     return 0;
53 }

```

```

Enter Name: lia
Enter Age: 45
Enter Roll No: 23
Enter Employee ID: 678

--- Intern Details ---
Name: lia
Age: 45
Roll No: 23
Employee ID: 678

```

Task2. School Management System

Design a program to represent a school management system using hybrid inheritance. • Base class: School (attribute: schoolName)

- Derived class 1: Teacher (attribute: subject)
- Derived class 2: Student (attribute: grade)
- Derived class from both Teacher and Student: TeachingAssistant Display information for all entities.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class School {
public:
    string schoolName;
    void getSchool() {
        cout << "Enter School Name: ";
        cin >> schoolName;
    }
};

class Teacher : virtual public School {
public:
    string subject;
    void getTeacher() {
        cout << "Enter Subject: ";
        cin >> subject;
    }
};

class Student : virtual public School {
public:
    string grade;
    void getStudent() {
        cout << "Enter Grade: ";
        cin >> grade;
    }
};

class TeachingAssistant : public Teacher, public Student {
public:
    void show() {
        cout << "\n--- TA Info ---" << endl;
        cout << "School: " << schoolName << endl;
        cout << "Subject: " << subject << endl;
    }
};
```

```

        cout << "Grade: " << grade << endl;
    }
};

int main() {
    TeachingAssistant ta;
    ta.getSchool();
    ta.getTeacher();
    ta.getStudent();
    ta.show();
    return 0;
}

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class School {
6  public:
7      string schoolName;
8      void getSchool() {
9          cout << "Enter School Name: ";
10         cin >> schoolName;
11     }
12 };
13
14 class Teacher : virtual public School {
15 public:
16     string subject;
17     void getTeacher() {
18         cout << "Enter Subject: ";
19         cin >> subject;
20     }
21 };
22
23 class Student : virtual public School {
24 public:
25     string grade;
26     void getStudent() {
27         cout << "Enter Grade: ";
28         cin >> grade;
29     }
30 };
31
32 class TeachingAssistant : public Teacher, public Student {
33 public:
34     void show() {

```

```

32 class TeachingAssistant : public Teacher, public Student {
33 public:
34     void show() {
35         cout << "\n--- TA Info ---" << endl;
36         cout << "School: " << schoolName << endl;
37         cout << "Subject: " << subject << endl;
38         cout << "Grade: " << grade << endl;
39     }
40 };
41
42 int main() {
43     TeachingAssistant ta;
44     ta.getSchool();
45     ta.getTeacher();
46     ta.getStudent();
47     ta.show();
48     return 0;
49 }

```

OUTPUT:

```

Enter School Name: SST
Enter Subject: Maths
Enter Grade: A-

```

```

--- TA Info ---
School: SST
Subject: Maths
Grade: A-

```

```

Enter School Name: SST
Enter Subject: Maths
Enter Grade: A-

--- TA Info ---
School: SST
Subject: Maths
Grade: A-

```

Task3. Vehicle Management System

Implement a system to manage different types of vehicles using hybrid inheritance.

- Base class: Vehicle (attribute: vehicleType)
- Derived class 1: Car
- Derived class 2: Bike
- From Car, derive ElectricCar
- From Bike, derive ElectricBike Each class should override a method to display specific features.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Vehicle {
public:
    string type;
    void show() {
        cout << "Vehicle Type: " << type << endl;
    }
};
```

```
class Car : public Vehicle {
public:
    void show() {
        type = "Car";
        cout << "This is a " << type << endl;
    }
};
```

```
class Bike : public Vehicle {
public:
    void show() {
        type = "Bike";
        cout << "This is a " << type << endl;
    }
};
```

```
class ElectricCar : public Car {
public:
    void show() {
        cout << "This is an Electric Car (Battery Powered)" << endl;
    }
};
```

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Vehicle {
6  public:
7      string type;
8      void show() {
9          cout << "Vehicle Type: " << type << endl;
10     }
11 };
12
13 class Car : public Vehicle {
14 public:
15     void show() {
16         type = "Car";
17         cout << "This is a " << type << endl;
18     }
19 };
20
21 class Bike : public Vehicle {
22 public:
23     void show() {
24         type = "Bike";
25         cout << "This is a " << type << endl;
26     }
27 };
28
29 class ElectricCar : public Car {
30 public:
31     void show() {
32         cout << "This is an Electric Car (Battery Powered)" << endl;
33     }
34 };
```

```
};

class ElectricBike : public Bike {
public:
    void show() {
        cout << "This is an Electric Bike (Battery
Powered)" << endl;
    }
};

int main() {
    ElectricCar ec;
    ElectricBike eb;
    ec.show();
    eb.show();
    return 0;
}
```

```
35
36 class ElectricBike : public Bike {
37 public:
38     void show() {
39         cout << "This is an Electric Bike (Battery Powered)" << endl;
40     }
41 };
42
43 int main() {
44     ElectricCar ec;
45     ElectricBike eb;
46     ec.show();
47     eb.show();
48     return 0;
49 }
```

OUTPUT:

This is an Electric Car (Battery Powered)
This is an Electric Bike (Battery Powered)

```
This is an Electric Car (Battery Powered)
This is an Electric Bike (Battery Powered)
```

Task4. Food Menu System

Create a food menu program using hybrid inheritance.

- Base class: Meal
- Derived classes: Breakfast, Lunch, and Dessert (inherit from Meal)
- From Breakfast, derive Juice, and from Dessert, derive Cake Each class should display a custom message using the print() function.

SOURCE CODE:

```
#include <iostream>
using namespace std;

class Meal {
public:
    void print() {
        cout << "This is a Meal." << endl;
    }
};
```

```
class Breakfast : public Meal {
public:
    void print() {
        cout << "Breakfast is the first meal." << endl;
    }
};

class Lunch : public Meal {
public:
    void print() {
        cout << "Lunch is the midday meal." << endl;
    }
};

class Dessert : public Meal {
public:
    void print() {
        cout << "Dessert is something sweet." << endl;
    }
};

class Juice : public Breakfast {
public:
    void print() {
        cout << "Juice is a fresh breakfast drink." << endl;
    }
};

class Cake : public Dessert {
public:
    void print() {
        cout << "Cake is a yummy dessert." << endl;
    }
};

int main() {
    Juice j;
    Cake c;
    Lunch l;

    j.print();
    l.print();
    c.print();

    return 0;
}
```

```

1  #include <iostream>
2  using namespace std;
3
4  class Meal {
5  public:
6      void print() {
7          cout << "This is a Meal." << endl;
8      }
9  };
10
11 class Breakfast : public Meal {
12 public:
13     void print() {
14         cout << "Breakfast is the first meal." << endl;
15     }
16 };
17
18 class Lunch : public Meal {
19 public:
20     void print() {
21         cout << "Lunch is the midday meal." << endl;
22     }
23 };
24
25 class Dessert : public Meal {
26 public:
27     void print() {
28         cout << "Dessert is something sweet." << endl;
29     }
30 };
31
32 class Juice : public Breakfast {
33 public:
34     void print() {

```

```

32     class Juice : public Breakfast {
33     public:
34         void print() {
35             cout << "Juice is a fresh breakfast drink." << endl;
36         }
37     };
38
39     class Cake : public Dessert {
40     public:
41         void print() {
42             cout << "Cake is a yummy dessert." << endl;
43         }
44     };
45
46     int main() {
47         Juice j;
48         Cake c;
49         Lunch l;
50
51         j.print();
52         l.print();
53         c.print();
54
55         return 0;
56     }

```

OUTPUT:

Juice is a fresh breakfast drink.

Lunch is the midday meal.

Cake is a yummy dessert.

```

Juice is a fresh breakfast drink.
Lunch is the midday meal.
Cake is a yummy dessert.

```